CHAPTER 2: THE PRINCIPLES OF OBJECT ORIENTED PROGRAMMING

OBJECT, CLASSES, ACCESS.

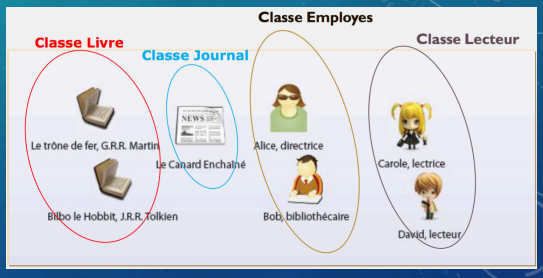
OBJECT

- An entity manipulated in a program (by calling methods)
- An object is characterized by:
  - Its identity: Unique
  - Its type
  - Its state: attribute value sat a given time
  - Its behavior: set of methods (consultation, update)



CLASS

- A family of objects with the same structure and behavior, characterized by a name.
- Each object belongs to a class.
- Allows you to instantiate a multitude of objects.
- Writing convention: *ClassName objectA*



CLASS
EXAMPLE

Person.java

```
public class Person {

public String Name, LastName;
public int age;
public void setName(String aName)
{
        Name = aName;
}
public String getName()
{
        return(Name );
}
}
```

Nom

Attributs

Methods

Person
Name: String LastName: String age: Integer
getName() setName()

Nom

Attributs

Methods

CLASS
DECLARATION OF ATTRIBUTES

Person.java

```
public class Person {

public String Name, LastName;
public int age;
public void setName(String aName)
{
        Name = aName;
}
public String getName()
{
        return (Name);
}
}
```

Attributes are "global" variables at the syntactic unit that constitutes the class: they are accessible in all methods of the class.

39

CLASS
DECLARATION OF METHODS

Person.java

```
public class Person {

public String Name, LastName;
public int age;
public void setName(String aName)
{
        Name = aName;
}
public String getName()
{
        return (Name);
}
}
```

Method declaration:
<return type> methodName(<parameter list>) {
<method body>
}

40

CLASS
DECLARATION OF METHODS

<return type>

```
<return type> methodName(<parameter list>) {
        <method body>
}
```

- When the method returns a value (function), specify the type of the returned value (simple type or class name):
  - double min(double a, double b)
  - int[] first(int n)
  - Color getColor()
- **void** if the method does not return a value : ***void*** *display(double[][] m)*

CLASS
DECLARATION OF METHODS

(<parameter list>)

```
<return type> methodName(<parameter list>) {
        <method body>
}
```

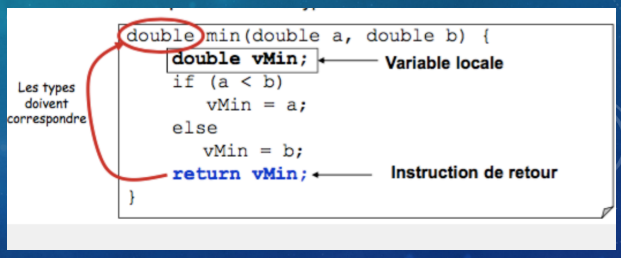
- Empty if the method has no parameters
  - int ReadInteger()
  - void display()
- a comma-separated sequence of *type-identifier* pairs
  - double min(double a, double b)
  - int min(int[] tab)
  - void setColor(color c)

CLASS
DECLARATION OF METHODS

<method body>

- A series of local variable declarations and statements.
- If the method has a return type, the method body must contain at least one `*return expression*` statement where `*expression*` returns a value compatible with the declared return type.

```
<return type> methodName(<parameter list>) {
        <method body>
}
```



CLASS
DECLARATION OF METHODS

<method body>

- If the method has a return type, the method body must contain at least one `return expression` statement...
- It is possible to have multiple `return` statements.

```
boolean contient(int[] tab, int val) {
    boolean trouve = false;
    int i = 0;
    while ((i < tab.length) && (! trouve)) {
            if (tab[i] == val)
                trouve = true;
        i++;
    }
    return trouve;
}
```

```
for (int i = 0; i < tab.length; i++)
   if (tab[i] == val)
     return true;
}
return false;
```

CLASS
DECLARATION OF METHODS

- ***return*** is also used to exit a method without returning a value (method with ***void*** as its return type).

```
void afficherPosition(int[] tab, int val) {

  for (int i = 0; i < tab.length; i++)
     if (tab[i] == val){
       System.out.println("La position de " + val + " est " + i);
       return;
     }

  System.out.println(val + " n'est pas présente dans le tableau");

}
```


CLASS
DECLARATION OF METHODS

<method body>

- A sequence of local variable declarations and instructions
- ***Local variables*** are variables declared inside a method.
  - They store the data manipulated by the method.
  - They are only accessible within the block in which they were declared.
  - Their value is lost when the method finishes executing.

CLASS
DECLARATION OF METHODS

<method body>

```
void method1(...) {
    int i;
    double y;
    int[] tab;
    ...
}
```

```
double method2(...) {
    double x;
    double y;
    double[] tab;
    ...
}
```

It is possible to use the same identifier in two separate methods without conflict; the local declaration is used in the method body.

CLASS

```
class Point {
   double x;
   double y;

   void translater(double dx,double dy){
         x += dx;
         y += dy;
   }

   double distance() {
         double dist;
         dist = Math.sqrt(x*x+y*y);
         return dist;
   }
}
```

```
class Cercle {
   double x;  // abscisse du centre
   double y;  // ordonnée du centre
   double r;  // rayon

   void translater(double dx,double dy){
         x += dx;
         y += dy;
   }

   ...
}
```

Two different classes can have members with the same name.

Members of a class are accessed via objects. Depending on the object's type (Point or Circle), Java can distinguish which attributes or methods are being referenced.

OBJECT

An object is an ***instance*** of a (single) class:

- it conforms to the description provided by that class,
- it has a unique value for each attribute declared in the class,
- these values characterize the object's state,
- any operation (***method***) defined in the class can be applied to it.

Every object has an identity that fully distinguishes it from other objects:

- it can be named and ***referenced*** by a name

CREATION OF OBJECTS (INSTANTIATION)

- To create (instantiate = reference) an object, you must:
  1. Declare a variable of type reference (of type class)
  2. Use the keyword "new" to allocate memory space
  3. Call a special method called a constructor, which allows the creation and initialization of an object (we will discuss this later)

Rules:

Object_Type Object_Name = new Constructor (parameter)

INSTANTIATION

Example:

MyClass myObject = new MyClass();

- `***new***` will reserve the necessary memory space to create the object "my object" of the class "***MyClass***".
- Example instances of the Person class:

```
//Declaration of Person type objects
Person John, Peter, X;
//Creation of objects
John = new Person();
Peter = new Person();
X = new Person();
```